

QUESTION 11: Create a class called Invoice that a hardware store might use to represent an invoice for an item sold at the store. An Invoice should include four pieces of information as instance. Data Members - • partNumber (type String) • partDescription (type String) • quantity of the item being purchased (type int) • price_per_item (type double) • Your class should have a constructor that initializes the four instance variables. Provide a set and a get method for each instance variable. In addition, provide a method named getInvoiceAmount() that calculates the invoice amount (i.e., multiplies the quantity by the price per item), and then returns the amount as a double value. If the quantity is not positive, it should be set to 0. If the price per item is not positive, it should be set to 0.0. Write a test application named invoiceTest that demonstrates class Invoice's capabilities.

Source code:

```
#include <iostream>

using namespace std;

class Invoice
{
    string partNumber;
    string partDescription;
    int quantity;
    double price_per_item;
public:
    Invoice() : partNumber("not assigned"),partDescription("not assigned"),
    quantity(0), price_per_item(0) {}
    void setter()
    {
        cout << "Enter the part number: ";
        cin >> partNumber;
        cout << "Enter the part description: ";
        cin >> partDescription;
        cout << "Enter the quantity: ";
        cin >> quantity;
        cout << "Enter the price per item: ";
        cin >> price_per_item;
    }
    void getvalue()
    {
```

```

cout << "\n*****The Item details are *****" << endl;

cout << "The part number is: " << partNumber << endl;

cout << "The part Description is: " << partDescription << endl;

cout << "The part Quantity is: " << quantity << endl;

cout << "The price per item is: " << price_per_item << endl;

}

double getInvoiceAmount()

{

double amount = 0;

amount = (quantity * price_per_item);

if (amount < 0)

return 0;

return amount;

}

};

int main()

{

Invoice o1;

o1.setter();

o1.getvalue();

double amount = o1.getInvoiceAmount();

cout << "The amount of Invoice is: " << o1.getInvoiceAmount();

return 0;

}

```

```

PS C:\Users\yogit\OneDrive\Desktop\DSA> cd "c:\Users\yogit\OneDrive\Desktop\DSA" ; if ($?) { g++ yogitajosi_c1_74.cpp -o yogitajosi_c1_74 } ; if ($?) {
.\yogitajosi_c1_74 }
Enter the part number: scr234
Enter the part description: mobile
Enter the quantity: 50
Enter the price per item: 259.50

*****The Item details are *****
The part number is: scr234
The part Description is: mobile
The part Quantity is: 50
The price per item is: 259.5
The amount of Invoice is: 12975
PS C:\Users\yogit\OneDrive\Desktop\DSA>

```

QUESTION 12: Imagine a tollbooth with a class called TollBooth. The two data items are of type unsigned int and double to hold the total number of cars and the total amount of money collected. A constructor initializes both of these data members to 0. A member function called payingCar() increments the car total and adds 0.5 to the cash total. Another function called nonPayCar() increments the car total but adds nothing to the cash total. Finally, a member function called display() shows the two totals. Include a program to test this class. This program should allow the user to push one key to count a paying car and another to count a non-paying car. Pushing the ESC key should cause the program to print out the total number of cars and total cash and then exit

Source code:

```
#include <iostream>

#include <conio.h>

using namespace std;

class tollBooth
{
private:
    unsigned int totalCars;
    double totalCash;

public:
    tollBooth() : totalCars(0), totalCash(0.0) {} // ---> constructor (initialisation list)

    void payingCar()
    {
        totalCars++;
        totalCash += 0.5;
    }

    void nonPayCar()
    {
        totalCars++;
    }

    void display() const
    {
        cout << "Total number of cars: " << totalCars << endl;
```

```

        cout << "Total cash collected: $" << totalCash << endl;
    }
};

int main()
{
    tollBooth booth;

    char key;

    cout << "Press 'p' for paying car, 'n' for non-paying car, 'Esc' to exit and display totals." << endl;

    while (true)
    {
        key = _getch();

        if (key == 27) // ---> ESC
        {
            booth.display();

            break;
        }

        else if (key == 'p' || key == 'P')
        {
            booth.payingCar();

            cout << "Paying car registered." << endl;
        }

        else if (key == 'n' || key == 'N')
        {
            booth.nonPayCar();

            cout << "Non-paying car registered." << endl;
        }

        else
        {
            cout << "Invalid input. Press 'p' for paying car, 'n' for non-paying car, or 'Esc' to exit." << endl;
        }
    }
}

```

```
return 0;
```

```
}
```

```
PS C:\Users\ayush\Desktop\lab journal> cd "c:\Users\ayush\Desktop\lab journal\" ; if ($?) { g++ yogita_joshi_74_c1.cpp -o yogita_joshi_74_c1
} ; if ($?) { .\yogita_joshi_74_c1 }
Press 'p' for paying car, 'n' for non-paying car, 'Esc' to exit and display totals.
Paying car registered.
Paying car registered.
Paying car registered.
Non-paying car registered.
Non-paying car registered.
Invalid input. Press 'p' for paying car, 'n' for non-paying car, or 'Esc' to exit.
Total number of cars: 5
Total cash collected: $1.5
PS C:\Users\ayush\Desktop\lab journal> █
```

Question 13: Create a class called Time that has separate int member data for hours, minutes, and seconds. One constructor should initialize this data to 0, and another should initialize it to fixed values. A member function should display it in 11:59:59 format. A member function named add() should add two objects of type time passed as arguments. A main() program should create two initialized values together, leaving the result in the third time variable. Finally, it should display the value of this third variable.

Source Code:

```
#include <iostream>

using namespace std;

class Time
{
private:
    int hour;
    int minute;
    int second;

public:
    Time()
    {
        hour = 0;
        minute = 0;
        second = 0;
    }
    Time(int h, int m, int s)
    {
        hour = h;
        minute = m;
        second = s;
    }
    void display()
    {
        cout << hour << ":" << minute << ":" << second << endl;
    }
}
```

```

void add(Time t1, Time t2)
{
    second = t1.second + t2.second;
    minute = t1.minute + t2.minute + (second / 60);
    hour = t1.hour + t2.hour + (minute / 60);
    second = second % 60;
    minute = minute % 60;
}
};

int main()
{
    Time t1(10, 200, 30);
    Time t2(15, 80, 15);
    Time t3;
    t3.add(t1, t2);
    t3.display();
    return 0;
}

```

```

PS C:\Users\yogit\OneDrive\Desktop\project\ygta> cd "c:\Users\yogit\OneDrive\Desktop\project\ygta\" ; if ($?) { g++ yogita.cpp -o yogita } ; if ($?) { .\yogita }
29:48:45
PS C:\Users\yogit\OneDrive\Desktop\project\ygta> 

```

QUESTION 14:

Construct a Program in C++ to show the working of function overloading(compile time polymorphism) by using a function named calculate Area () to calculate area of square, rectangle and triangle using different signatures as required.

Source code:

```
#include <iostream>

using namespace std;

float area(int s)
{
    return s * s;
}

float area(int l, int b)
{
    return l * b;
}

float area(float ba, float h)
{
    return (ba * h);
}

int main()
{
    int s, l, b, h, ba;

    cout << "Enter the side of the square: ";

    cin >> s;

    cout << "Enter the length and breath of the rectangle: ";

    cin >> l >> b;

    cout << "Enter the height and the base of the triangle: ";

    cin >> ba >> h;

    cout << "Area of square: " << area(s) << endl;

    cout << "Area of rectangle: " << area(l, b) << endl;

    cout << "Area of triangle: " << (area(ba, h) / 2) << endl;

    return 0;
}
```

}

```
roject\ygta\" ; if ($?) { g++ yogita.cpp -o yogita } ; if ($?) {  
Enter the side of the square: 4  
Enter the length and breath of the rectangle: 5 6  
Enter the height and the base of the triangle: 8 2  
Area of square: 16
```

QUESTION 15:

Implement a Program in C++ by defining a class to represent a bank account.

Include the following:

Data Members

- Name of the depositor
- Account number
- Type of account (Saving, Current etc.)
- Balance amount in the account

Member Functions

- To assign initial values
- To deposit an amount
- To withdraw an amount after checking the balance
- To display name and balance

Source Code:

```
#include <iostream>

using namespace std;

class bank
{
private:
    string name;
    int num;
    string type;
    float balance_amount;
    float deposit_amount;
    float total_amount;
    float withdraw_amount;
public:
```

```

void read();

void deposit();

void withdraw();

void display();

};

void bank::display(void)
{
    cout << "The name of the account holder is: " << name << endl;
    cout << "The balance left is: " << total_amount << endl;
}

void bank::read(void)
{
    cout << "Enter the name of the depositor: ";
    getline(cin,name);
    cout << "Enter the account number: ";
    cin >> num;
    cin.ignore();
    cout << "Enter the type of account(sav./curr.): ";
    getline(cin,type);
    cout << "Enter the balance amount: ";
    cin >> balance_amount;
}

void bank::deposit(void)
{
    cout << "Enter the amount to deposit: ";
    cin >> deposit_amount;
    total_amount = balance_amount + deposit_amount;
}

void bank::withdraw(void)
{
    cout << "The balance is: " << total_amount << endl;

```

```

cout << "Enter the amount to withdraw: ";

cin >> withdraw_amount;

if(withdraw_amount >= total_amount)
{
    cout << "insufficient balance." << endl;
}
else
{
    total_amount = total_amount - withdraw_amount;
}
}

int main()
{
    bank b;

    b.read();

    b.deposit();

    b.withdraw();

    b.display();

    return 0;
}

```

```

PS C:\Users\ayush\Desktop\lab journal> cd "c:\Users\ayush\Desktop\lab journal\" ; if ($?) { g++ yogita_joshi_74_c1.cpp -o yogita_joshi_74_c1
} ; if ($?) { .\yogita_joshi_74_c1 }
After setting annual interest rate to 4%:
Saver1 balance: Rs 2006.67
Saver2 balance: Rs 3010
After setting annual interest rate to 5%:
Saver1 balance: Rs 2015.03
Saver2 balance: Rs 3022.54
PS C:\Users\ayush\Desktop\lab journal>

```

QUESTION 16:

Create class SavingsAccount. Use a static variable annualInterestRate to store the annual interest rate for all account holders. Each object of the class contains a private instance variable savingsBalance indicating the amount the saver currently has on deposit. Provide method For batches admitted in 2023- 24 and 2024-25 onwards, Applicable from Academic Session 2024-25 calculateMonthlyInterest() to calculate the monthly interest by multiplying the savingsBalance by annualInterestRate divided by 12.This interest should be added to savingsBalance. Provide a static method modifyInterestRate() that sets the annualInterestRate to a new value. Write a program to test class SavingsAccount. Instantiate two savingsAccount objects, saver1 and saver2, with balances of Rs2000.00 and Rs3000.00, respectively. Set annualInterestRate to 4%, then calculate the monthly interest and print the new balances for both savers. Then set the annualInterestRate to 5%, calculate the next month's interest and print the new balances for both savers

SOURCE CODE:

```
#include <iostream>

using namespace std;

class SavingsAccount
{
private:
    double savingsBalance;
    static double annualInterestRate;

public:
    SavingsAccount(double balance)
    {
        savingsBalance = balance;
    }

    void calculateMonthlyInterest()
    {
        double monthlyInterest = (savingsBalance * annualInterestRate) / 12;
        savingsBalance += monthlyInterest;
    }

    static void modifyInterestRate(double newRate)
    {
        annualInterestRate = newRate;
    }
}
```

```

    }

    double getBalance() const
    {
        return savingsBalance;
    }
};

// Initialize the static member
double SavingsAccount::annualInterestRate = 0.0;

int main()
{
    SavingsAccount saver1(2000);
    SavingsAccount saver2(3000);
    SavingsAccount::modifyInterestRate(0.04);
    saver1.calculateMonthlyInterest();
    saver2.calculateMonthlyInterest();
    cout << "After setting annual interest rate to 4%:" << endl;
    cout << "Saver1 balance: Rs " << saver1.getBalance() << endl;
    cout << "Saver2 balance: Rs " << saver2.getBalance() << endl;
    SavingsAccount::modifyInterestRate(0.05);
    saver1.calculateMonthlyInterest();
    saver2.calculateMonthlyInterest();
    cout << "After setting annual interest rate to 5%:" << endl;
    cout << "Saver1 balance: Rs " << saver1.getBalance() << endl;
    cout << "Saver2 balance: Rs " << saver2.getBalance() << endl;
    return 0;
}

```

```

PS C:\Users\ayush\Desktop\lab journal> cd "c:\Users\ayush\Desktop\lab journal\" ; if ($?) { g++ yogita_joshi_74_c1.cpp -o yogita_joshi_74_c1
} ; if ($?) { .\yogita_joshi_74_c1 }
After setting annual interest rate to 4%:
Saver1 balance: Rs 2006.67
Saver2 balance: Rs 3010
After setting annual interest rate to 5%:
Saver1 balance: Rs 2015.03
Saver2 balance: Rs 3022.54

```

QUESTION 17:

Create a class Complex having two int type variable named real & img denoting real and imaginary part respectively of a complex number. Overload +, -, == operator to add, to subtract and to compare two complex numbers being denoted by the two complex type objects

SOURCE CODE:

```
#include <iostream>

using namespace std;

class Complex
{
    int real, img;

public:
    Complex(int real = 0, int img = 0)
    {
        this->real = real;
        this->img = img;
    }

    void display()
    {
        cout << "The number is: " << real << " + " << img << "i" << endl;
    }

    Complex operator+(Complex &o1)
    {
        Complex o;
        o.real = real + o1.real;
        o.img = img + o1.img;
        return o;
    }

    Complex operator-(Complex &o1)
    {
        Complex o;
```

```

        o.real = real - o1.real;

        o.img = img - o1.img;

        return o;
    }

    int operator==(Complex &o1)
    {
        return real == o1.real && img == o1.img;
    }
};

int main()
{
    Complex o1(2, 9);
    Complex o2(2, 9);
    Complex o3;
    o3 = o1 + o2;
    o3.display();
    o3 = o1 - o2;
    o3.display();
    if (o1 == o2)
        cout << "Equal\n";
    else
        cout << "unequal\n";

    return 0;
}

```

```

PS C:\Users\yogit\OneDrive\Desktop\DSA> cd "c:\Users\yogit\OneDrive\Desktop\DSA\" ; if ($?) { g++ yogitajosi_c1_74.cpp -o yogitajosi_c1_74 } ; if ($?) { .\yogitajosi_
C1_74 }
Enter the first complex number:
Enter a real number: 55
Enter an imaginary number: 10i
Enter the second complex number:
Enter a real number: Enter an imaginary number: Sum: 55 + 10i
Subtraction: 55 + 10i
Complex numbers are not equal.
PS C:\Users\yogit\OneDrive\Desktop\DSA> █

```

QUESTION 18:

Using the concept of operator overloading. Implement a program to overload the following:

- a. Unary –
- b. Unary ++ preincrement, postincrement
- c. Unary -- predecrement, postdecrement

SOURCE CODE:

```
#include <iostream>

using namespace std;

class unary
{
    int a, b;

public:
    unary(int a = 0, int b = 0)
    {
        this->a = a;
        this->b = b;
    }

    // ---> preIncrement operator overloading for unary class
    unary operator++()
    {
        unary o;
        o.a = ++a;
        o.b = ++b;
        return o;
    }
}
```

// ---> preDecriment operator overloading for unary class

unary operator--()

```
{  
    unary o;  
    o.a = --a;  
    o.b = --b;  
    return o;  
}
```

// ---> postIncrement operator overloading for unary class

unary operator++(int)

```
{  
    unary o;  
    o.a = a;  
    o.b = b;  
    a++;  
    b++;  
    return o;  
}
```

// ---> postDecriment operator overloading for unary class

unary operator--(int)

```
{  
    unary o;  
    o.a = a;  
    o.b = b;  
    a--;  
    b--;  
    return o;  
}
```

void display()

```
{  
    cout << "The value of a is " << a << " and b is " << b << endl;  
}
```

```

    }
};

int main()
{
    unary o1(1, 2);
    unary o2(2, 3);
    unary o3 = o1++;
    o3.display();
    unary o4 = o1--;
    o4.display();
    unary o5 = ++o1;
    o5.display();
    unary o6 = --o1;
    o6.display();
    return 0;
}

```

```

> cd "c:\Users\yogit\OneDrive\Desktop\DSA\" ; if ($?) { g++ yogitajosi_C1_74.cpp -o yo
{ .\yogitajosi_C1_74 }
Initial Value: 10
After negation Value: -10
After pre-increment Value: 11
After post-increment Value: 12
After pre-decrement Value: 11
After post-decrement Value: 10
PS C:\Users\yogit\OneDrive\Desktop\DSA>

```